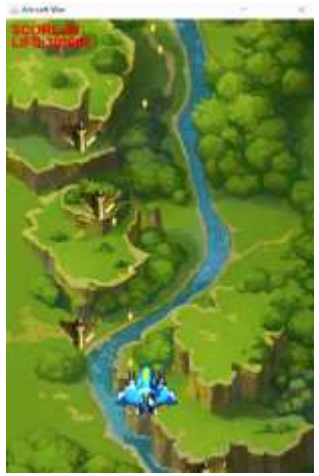
实验四：DAO模式和单元测试

实验与创新实践教育中心 • 计算机与数据技术实验教学部

本学期实验总体安排

初始版本



最终版本

初始版本

游戏主界面
英雄机移动
英雄机子弹直射
碰撞检测
统计得分和生命值

01

UML建模
精英敌机生成、攻击
道具生成、加血道具生效
采用**单例模式**创建英雄机

02

采用**简单工厂模式**
创建道具
采用**工厂方法模式**
创建敌机

03

采用**策略模式**
实现不同飞机的发射弹道
两种火力道具生效

04

采用**数据访问对象模式**
实现得分排行榜
使用**JUnit单元测试**

05

使用**Swing**添加游戏难度选择和排行榜界面
使用**多线程**实现音效控制及火力道具状态恢复

06

采用**观察者模式**
实现炸弹、冰冻道具
采用**模板模式**
实现三种游戏难度

本学期实验总体安排

实验项目	一	二	三	四	五	六
学时数	2	2	2	2	4	4
实验内容	系统分析 单例模式	简单工厂 工厂方法模式	策略模式	DAO模式 单元测试	Swing 多线程	观察者模式 模板模式
时间节点			**中期**			**结项**
提交内容			UML类图 项目代码			结项报告 代码、类图 展示视频

实验课程共 16 个学时，6 个实验项目，总成绩为 30分 。

目录

CONTENTS

01 实验目的

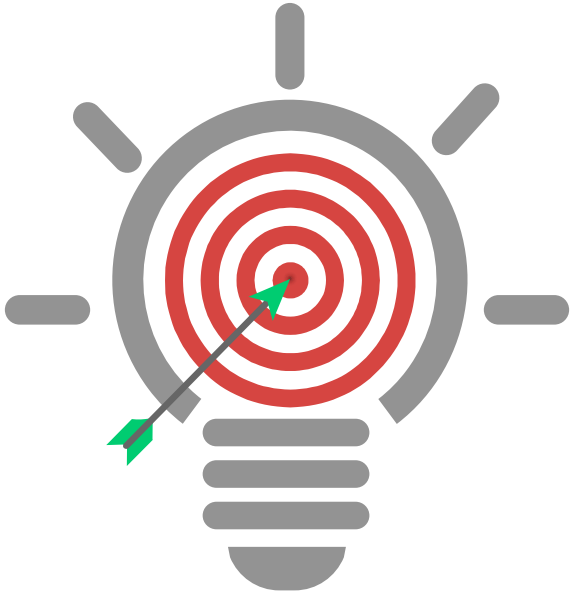
02 实验任务

03 实验步骤

04 迭代目标

实验目的

- 深入理解**数据访问对象（DAO）模式**的设计意图及结构
- 结合案例，规范绘制DAO模式的**UML类图**
- 掌握**重构**方法，通过**代码实现**并应用DAO模式
- 了解**单元测试**的定义及其重要性
- 掌握使用**JUnit5**进行单元测试的常用方法



实验任务

1. 深入理解DAO模式的设计原理与结构，并结合实例规范绘制该模式的 UML 结构图；
2. 完成代码重构，采用 DAO 模式实现玩家得分排行榜的相关功能（数据访问、存储与展示）；
3. 对英雄机类进行单元测试，至少选择 3 个方法（含其父类方法）作为测试对象，设计合理测试用例并完成测试；
4. 完成本次迭代开发清单中的全部任务。

实验步骤：排行榜数据持久化场景分析（1）

排行榜数据持久化场景分析

在飞机大战游戏中，系统实时记录英雄机的当前得分。每局游戏结束后，展示当前难度的总分排行榜，玩家可选择**保存**本局游戏记录或**删除**指定的历史记录。

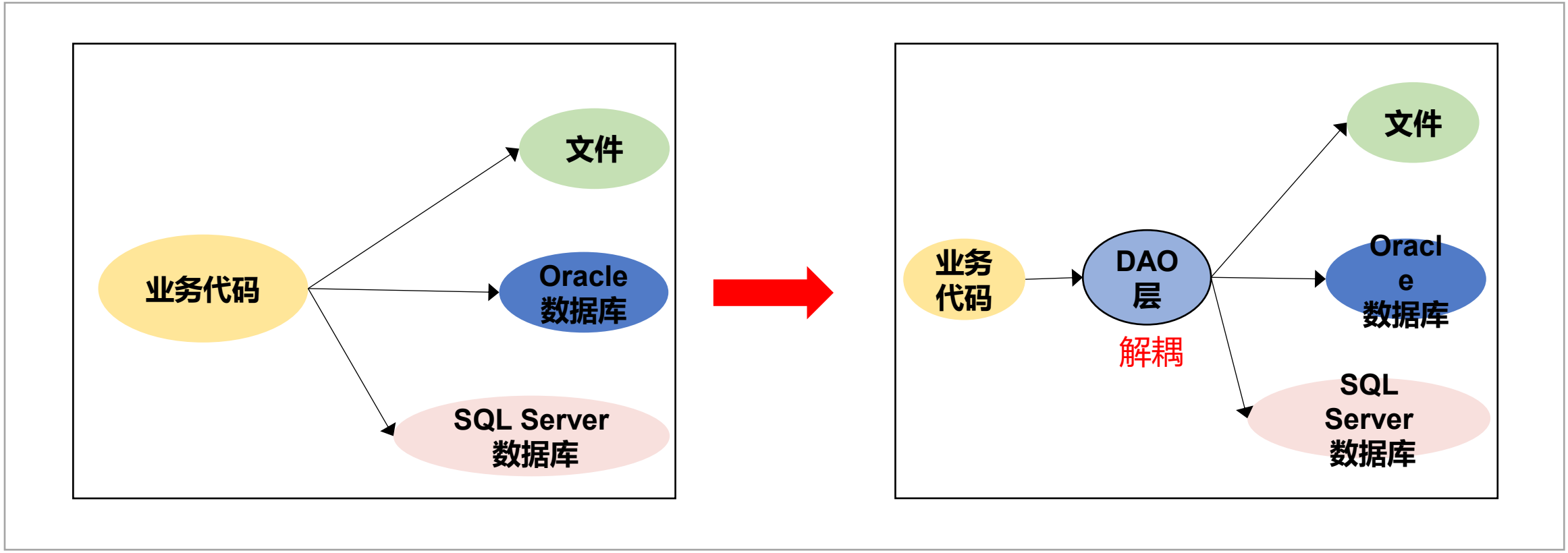
榜单内容包括：名次、玩家名、得分和时间。

（本次实验无需与用户交互，控制台输出即可）

实验步骤：排行榜数据持久化场景分析（2）

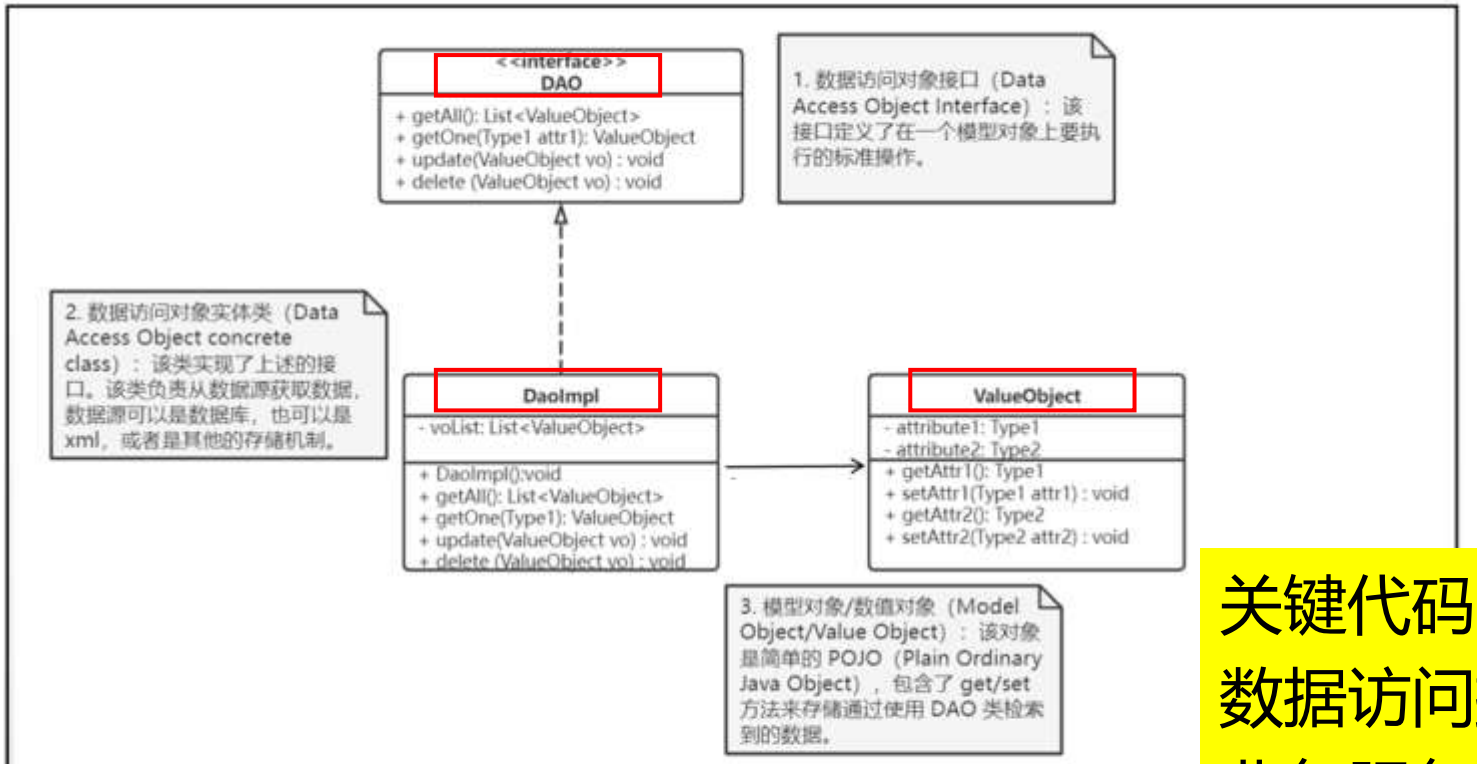
请思考： 1. 玩家得分数据存储在哪里？如何获取和修改玩家的得分？

2. 若更换数据源，需要改动哪些类的代码？



实验步骤：DAO模式结构图（1）

数据访问对象模式（Data Access Object Pattern）也叫做 DAO 模式，用于把低级的数据访问 API 或操作从高级的业务服务中分离出来。



数据访问对象模式结构图

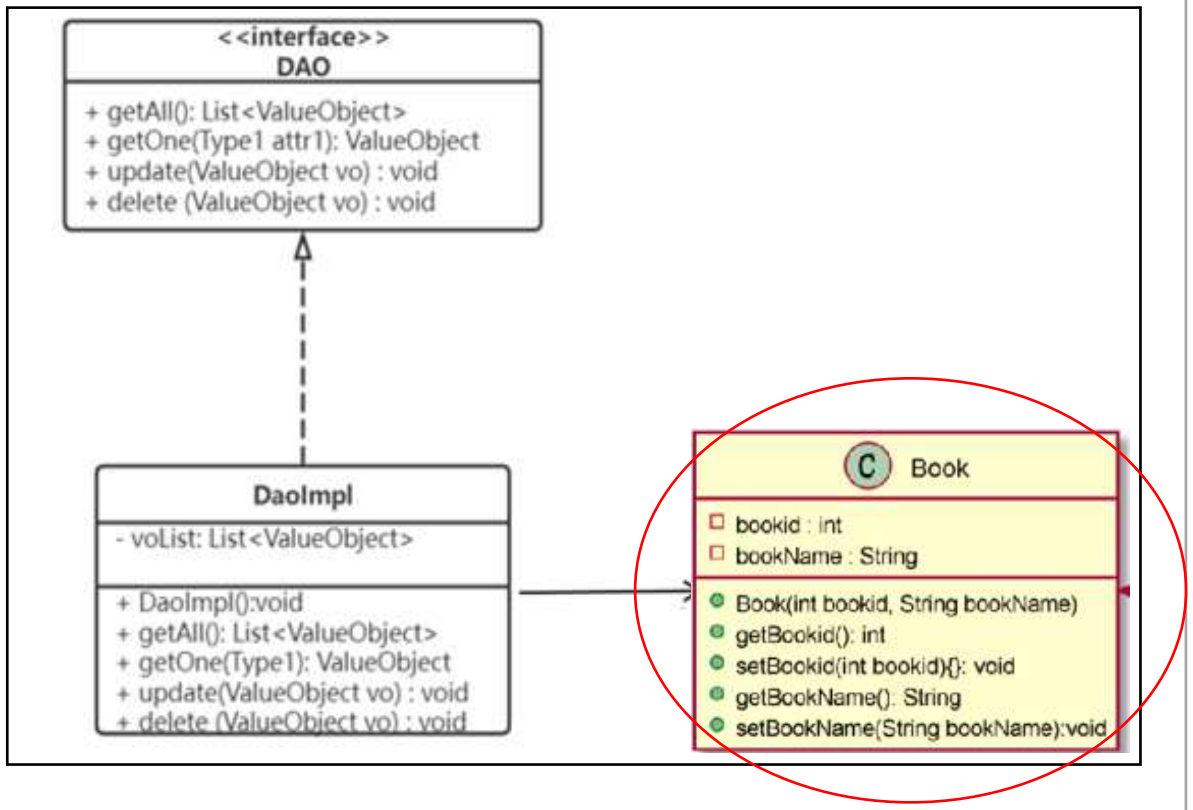
关键代码：分离低级的数据访问操作和高级的业务服务

实验步骤：图书管理系统举例（DAO模式）

假如我们实现一个图书管理系统，实现查询所有图书、按编号查询、增加、删除图书的功能。我们该如何绘制UML类图？

实验步骤：图书管理系统举例（1）

UML 类 图 设 计

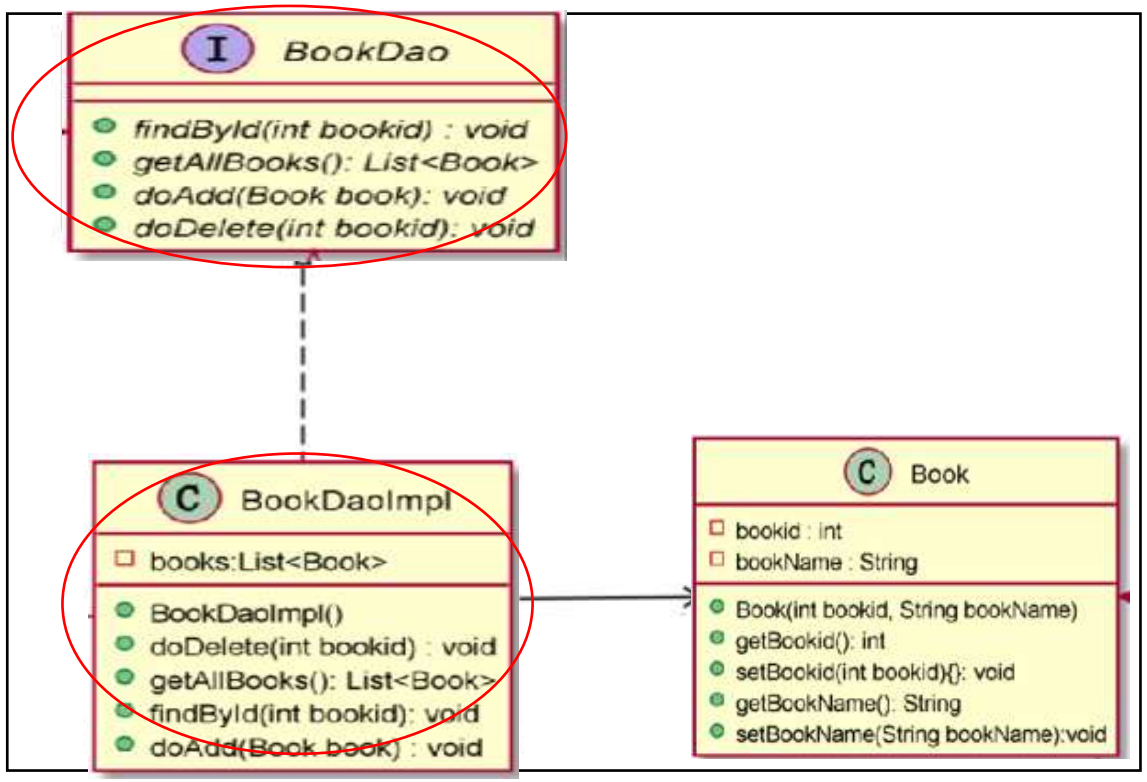


① 创建一个作为数值对象的实体类Book

```
public class Book {
    private int bookid;
    private String bookName;

    Book(int bookid, String bookName) {
        this.bookid = bookid;
        this.bookName = bookName;
    }

    public int getBookid() {
        return bookid;
    }
    public void setBookid(int bookid) {
        this.bookid = bookid;
    }
    public String getBookName() {
        return bookName;
    }
    public void setBookName(String bookName) {
        this.bookName = bookName;
    }
}
```

实验步骤：图书管理系统举例（2）

UML 类图设计



② 创建数据访问对象DAO接口

```java
public interface BookDao {

    void findById(int bookid);

    List<Book> getAllBooks();

    void doAdd(Book book);

    void doDelete(int bookid);
}
```

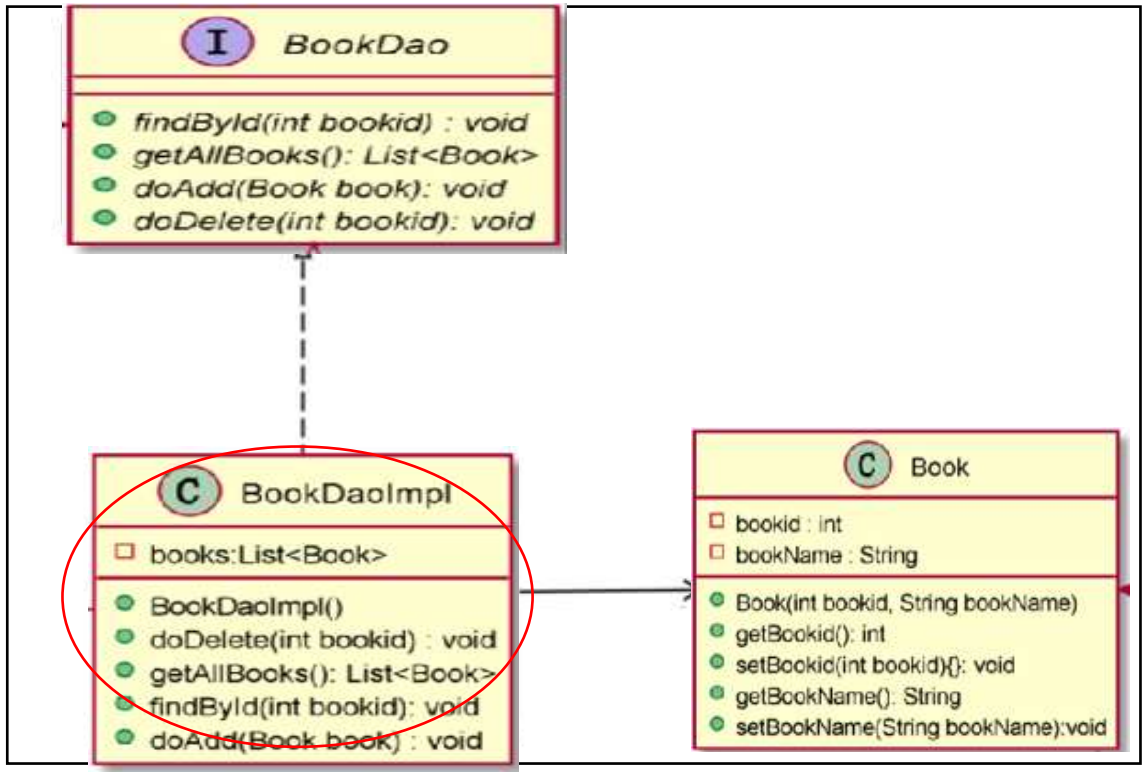
③ 创建实现了上述接口的DAO实现类

1

```java
public class BookDaoImpl implements BookDao {

    //模拟数据库数据
    private List<Book> books;

    public BookDaoImpl() {
        books = new ArrayList<Book>();
        books.add(new Book(1001, "Clean Code"));
        books.add(new Book(1002, "Design Patterns"));
        books.add(new Book(1003, "Effective Java"));
    }

    //获取所有图书
    @Override
    public List<Book> getAllBooks() {
        return books;
    }
```

实验步骤：图书管理系统举例（3）

UML 类 图 设 计



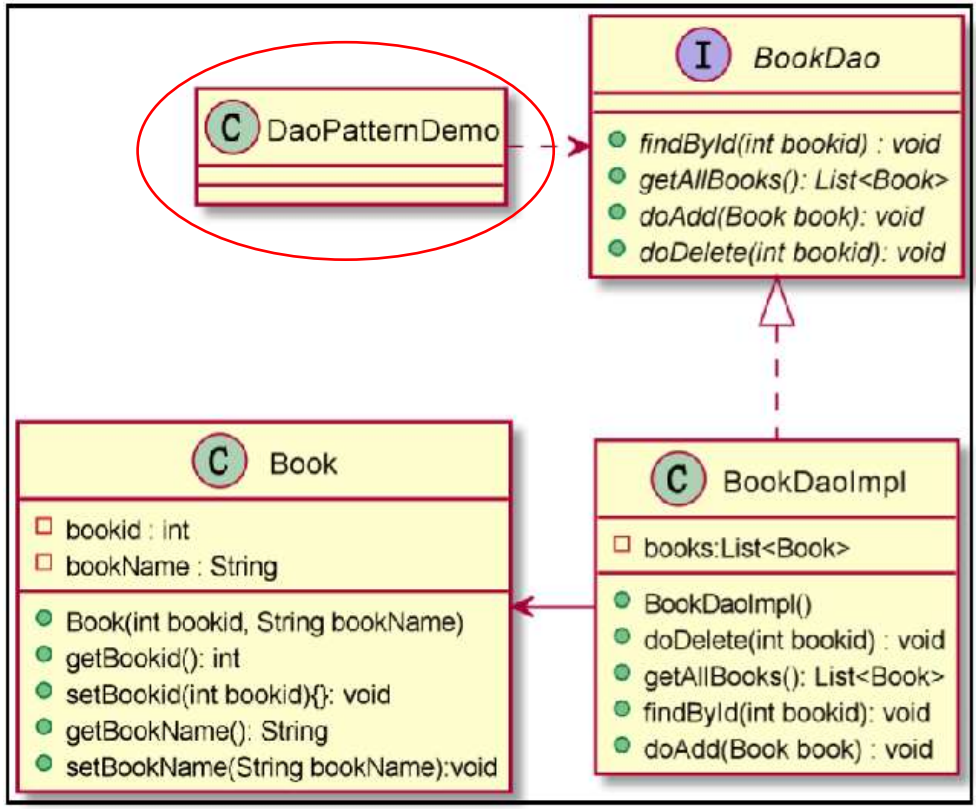
③ 创建实现了上述接口的DAO实现类

2

```
    //查找图书
    @Override
    public void findById(int bookid) {
        for (Book item : books) {
            if (item.getBookid() == bookid) {
                System.out.println("Find Book: ID [" + bookid +
                        "], Book Name [" + item.getBookName() + "]");
                return;
            }
        }
        System.out.println("Can not find this book!");
    }

    //删除图书
    @Override
    public void doDelete(int bookid) {
        for (Book item : books) {
            if (item.getBookid() == bookid) {
                books.remove(item);
                System.out.println("Delete Book: ID [" + bookid + "]");
                return;
            }
        }
        System.out.println("Can not find this book!");
    }

    //新增图书
    @Override
    public void doAdd(Book book) {
        books.add(book);
        System.out.println("Add new Book: ID [" + book.getBookid() +
                "], Book Name [" + book.getBookName() + "]");
    }
}
```

实验步骤：图书管理系统举例（4）

UML 类 图 设 计



④ 使用DaoPatternDemo来演示数据访问对象模式的用法

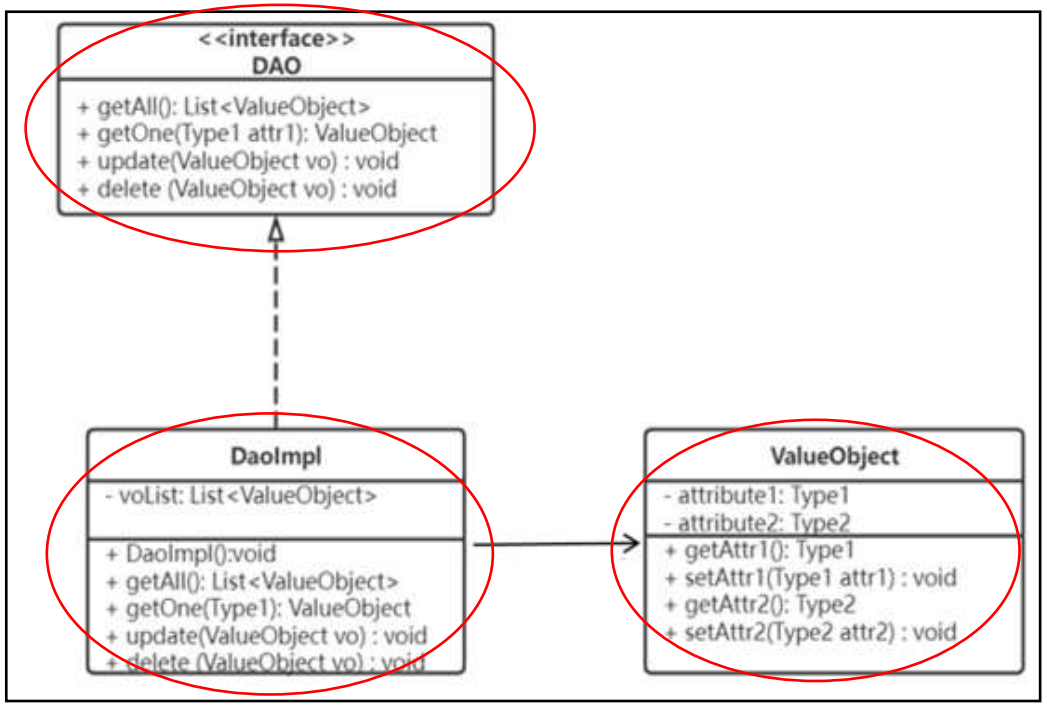
```
public class DaoPatternDemo {
    public static void main(String[] args) {

        BookDao bookDao = new BookDaoImpl();

        //输出所有图书
        for (Book book : bookDao.getAllBooks()) {
            System.out.println("Book ID [" + book.getBookid() +
                    "], Book Name : [" + book.getBookName() + "]");
        }

        //查找图书
        bookDao.findById(1002);

        //删除图书
        bookDao.doDelete(1002);

        //新增图书
        Book newB
        bookDao.d

        //输出所有
        for (Book
            Syste

        }
    }
}
```

```
Book ID [1001], Book Name : [Clean Code]
Book ID [1002], Book Name : [Design Patterns]
Book ID [1003], Book Name : [Effective Java]

Find Book: ID [1002], Book Name [Design Patterns]

Delete Book: ID [1002]

Add new Book: ID [1004], Book Name [Thinking In java]

Book ID [1001], Book Name : [Clean Code]
Book ID [1003], Book Name : [Effective Java]
Book ID [1004], Book Name : [Thinking In java]
```

实验步骤：DAO模式结构图（2）

请思考：

结合排行榜，数值对象实体类需要哪些属性？DAO接口需要提供哪些方法？



实验步骤：JUnit与单元测试

➢ 单元测试

- 是指对软件中的最小可测试单元进行检查和验证；
- 自动化测试，一般由程序员自己编写；
- 提升软件质量，增加重构自信。

➢ JUnit

- 一个Java 语言的单元测试框架；
- 促进了测试驱动开发的发展；
- 大部分的Java IDE都集成了JUnit作为单元测试工具；
- 官方文档：JUnit User Guide

实验步骤：如何用JUnit5进行单元测试（1）

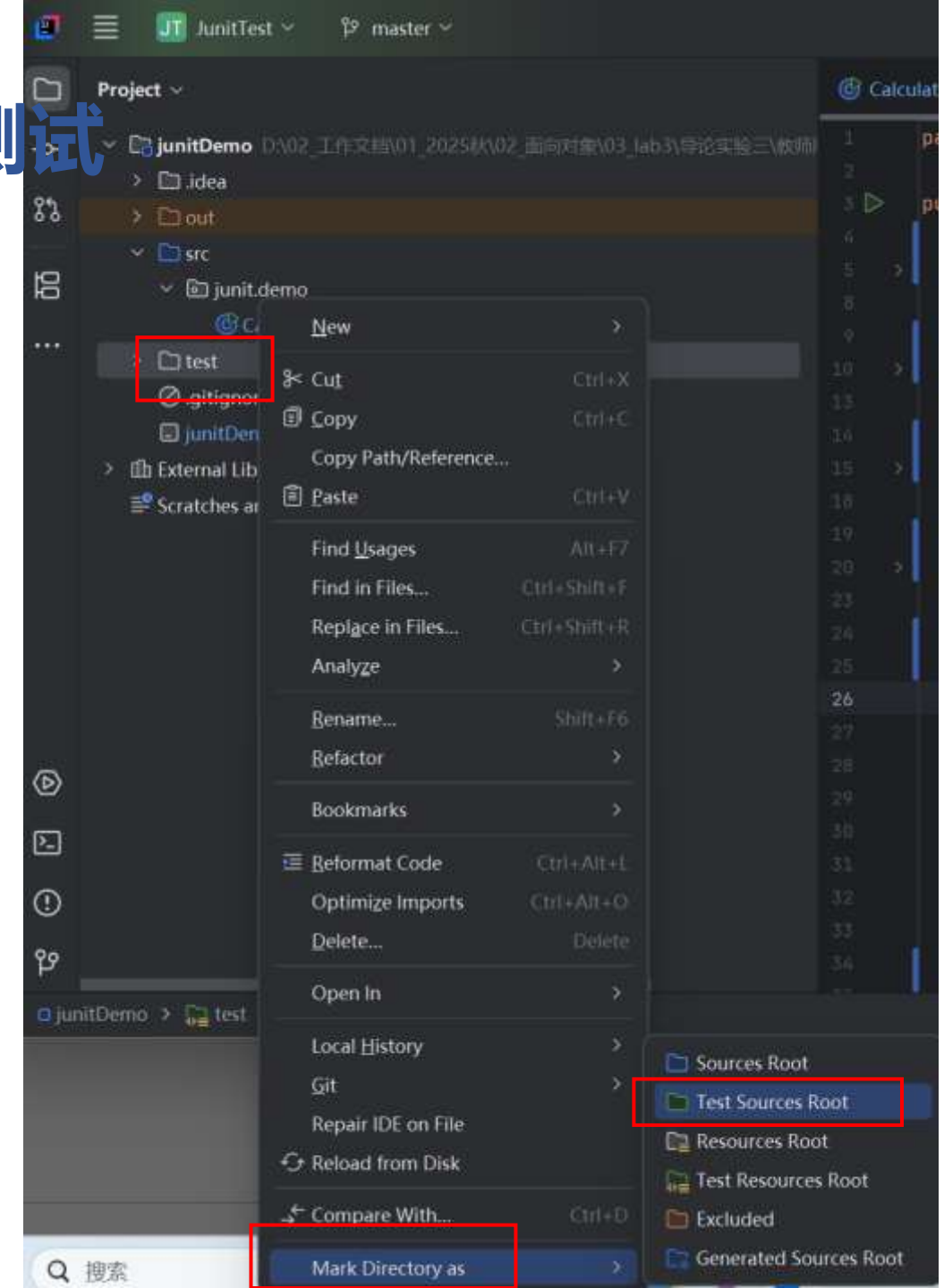
假如，有一个实现简易计算器功能的类Calculator，有加、减、乘、除功能。如何用JUnit5对它进行单元测试?

```java
public class Calculator {
    public int add(int x, int y) { //加法
        return x + y;
    }

    public int sub(int x, int y) { //减法
        return x - y;
    }

    public int mul(int x, int y) { //乘法
        return x * y;
    }

    public int div(int x, int y) { //除法
        return x / y;
    }

    public int div2(int x, int y) { //除法 做了异常判断
        try {
            int z = x / y;
        } catch (Exception e) {
            e.printStackTrace();
        }
        return x / y;
    }

    public void unCompleted(int x, int y) { //未完成的模块:例如平方、开方等等
        //TODO
    }
}
```

实验步骤：如何用JUnit5进行单元测试

①创建测试类文件夹

新建test文件夹，右键选择 Mark Directory as → Test Sources Root。



实验步骤：如何用JUnit5进行单元测试

② 创建JUnit单元测试类

在待测试的类中，按下快捷键ctrl + shift + T，选择Create New Test。

Calculator.java

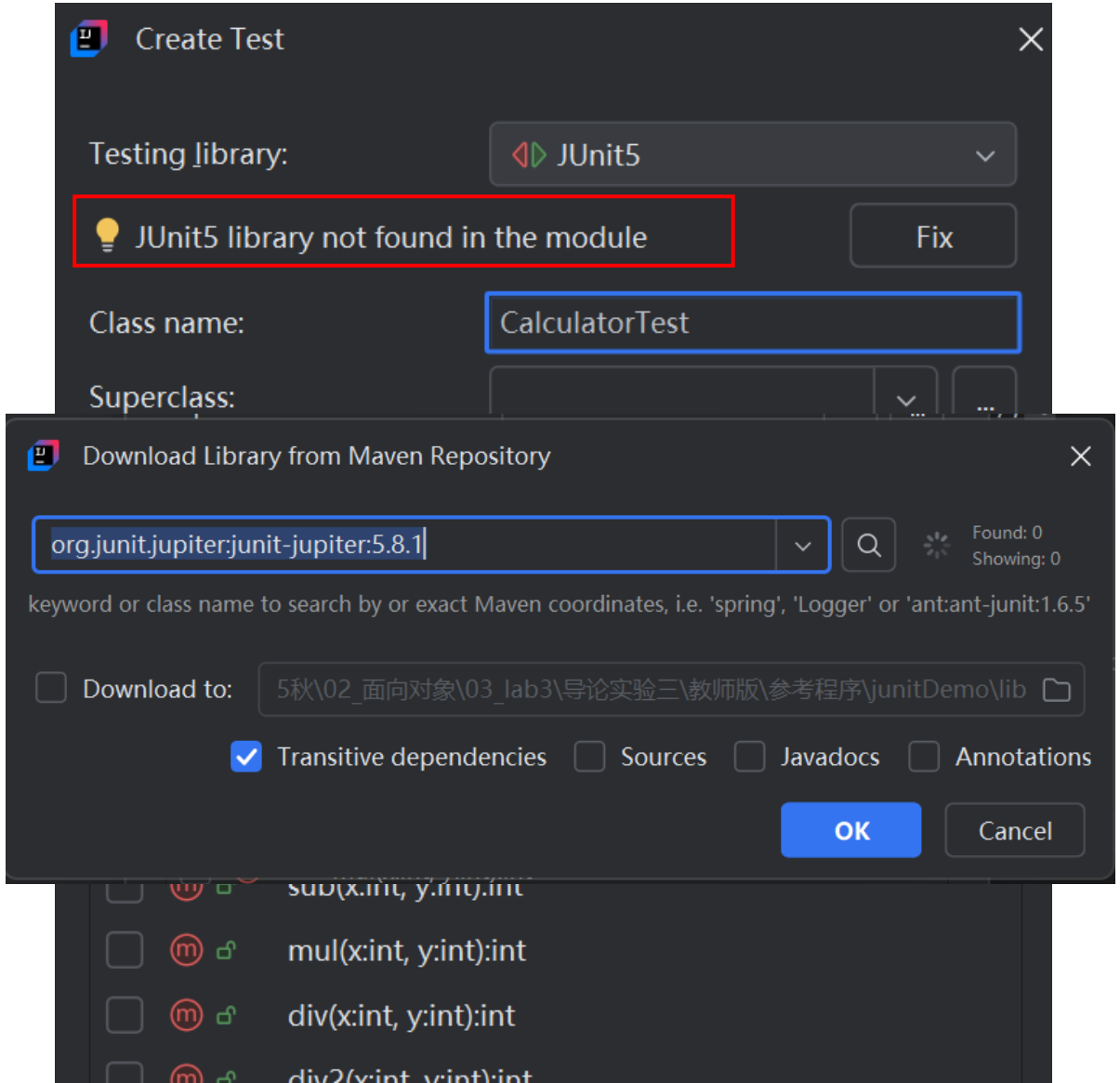
```
1   package junit.demo;
2
3   public class Calculator {   duskphantom *
4       //加法
5       public int add(int x, int y) { return x + y; }
8
9       //减法
10      public int sub(int x, int y) { return x - 1; }
13
14      //乘法
15      public int mul(int x, int y) { return x * y; }
18
19      //除法
20      public int div(int x, int y) { return x / y; }
23
24      //除法 做了异常判断
25      public int div2(                                  tom *
26          try {
27              int z =
28          } catch (Exception e) {
```

Choose Test for Calculator (0 found)

Create New Test...

实验步骤：如何用JUnit5进行单元测试

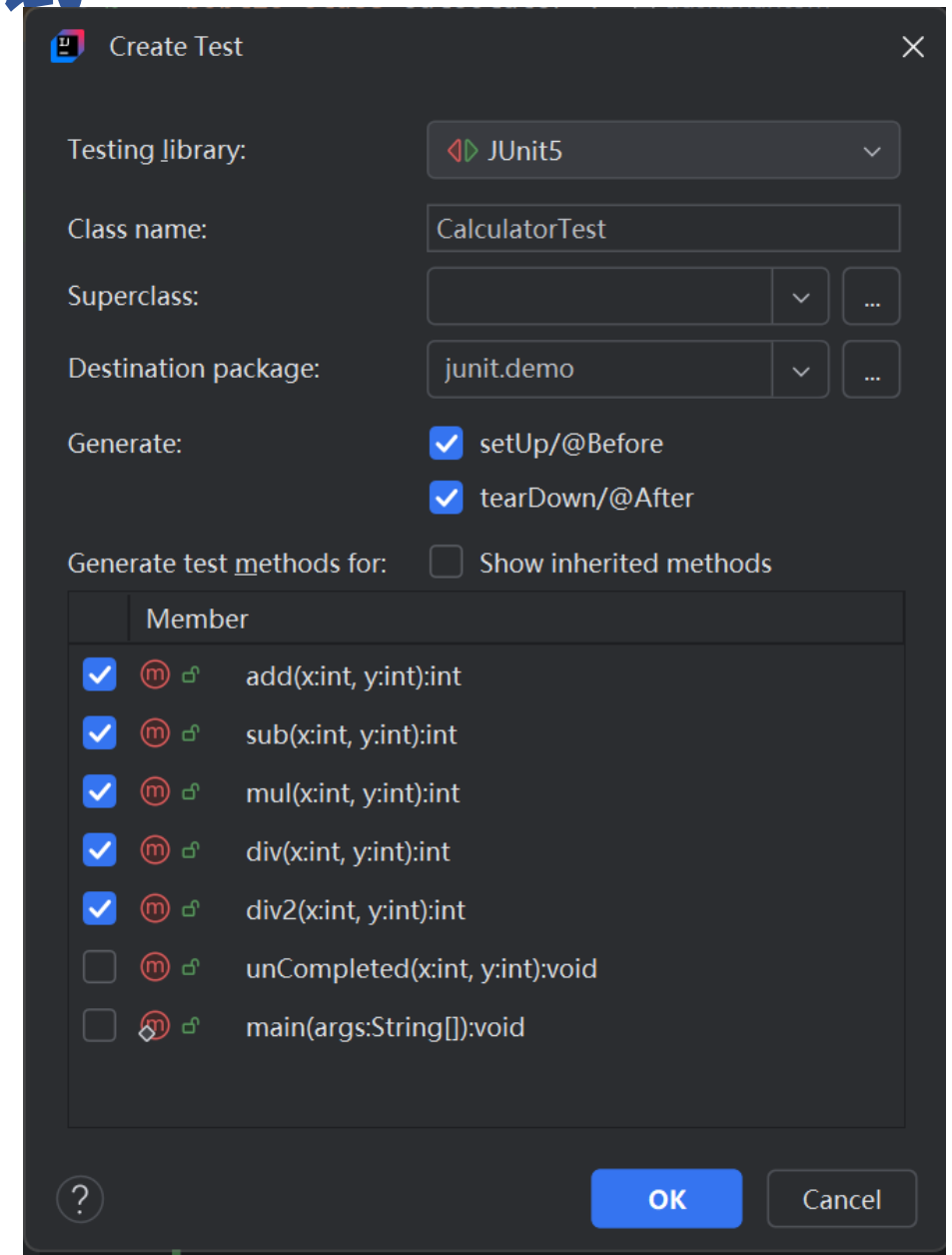
注意：
第一次使用JUnit5需要安装



实验步骤：如何用JUnit5进行单元测试

③ 勾选需要测试的方法

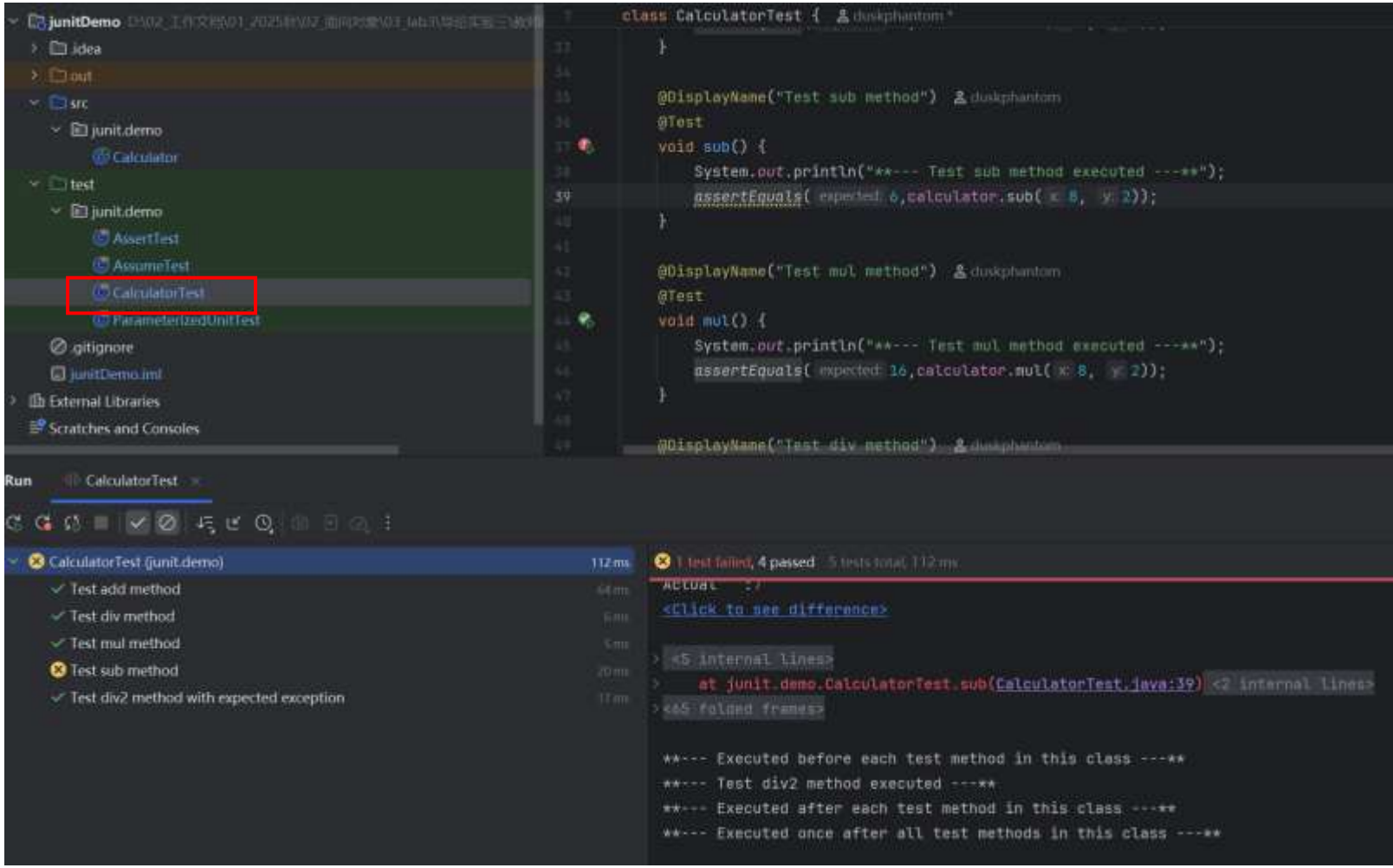
在Member中勾选Calculator 中需要进行单元测试的方法。



实验步骤：如何用JUnit5进行单元测试

④ 确认单元测试类生成

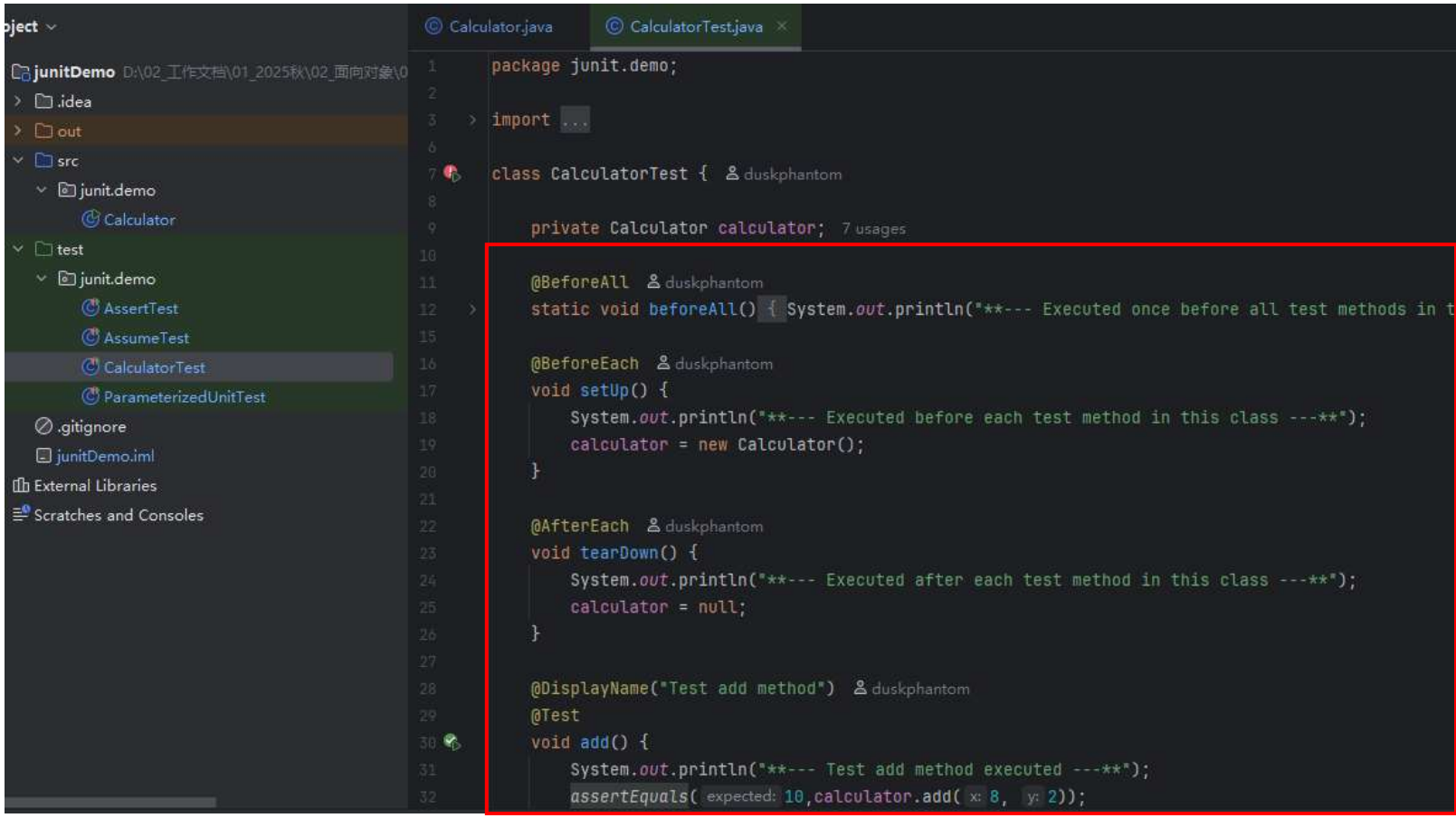
查看test目录下生成的单元测试类CalculatorTest。



实验步骤：如何用JUnit5进行单元测试

⑤ 编写单元测试代码

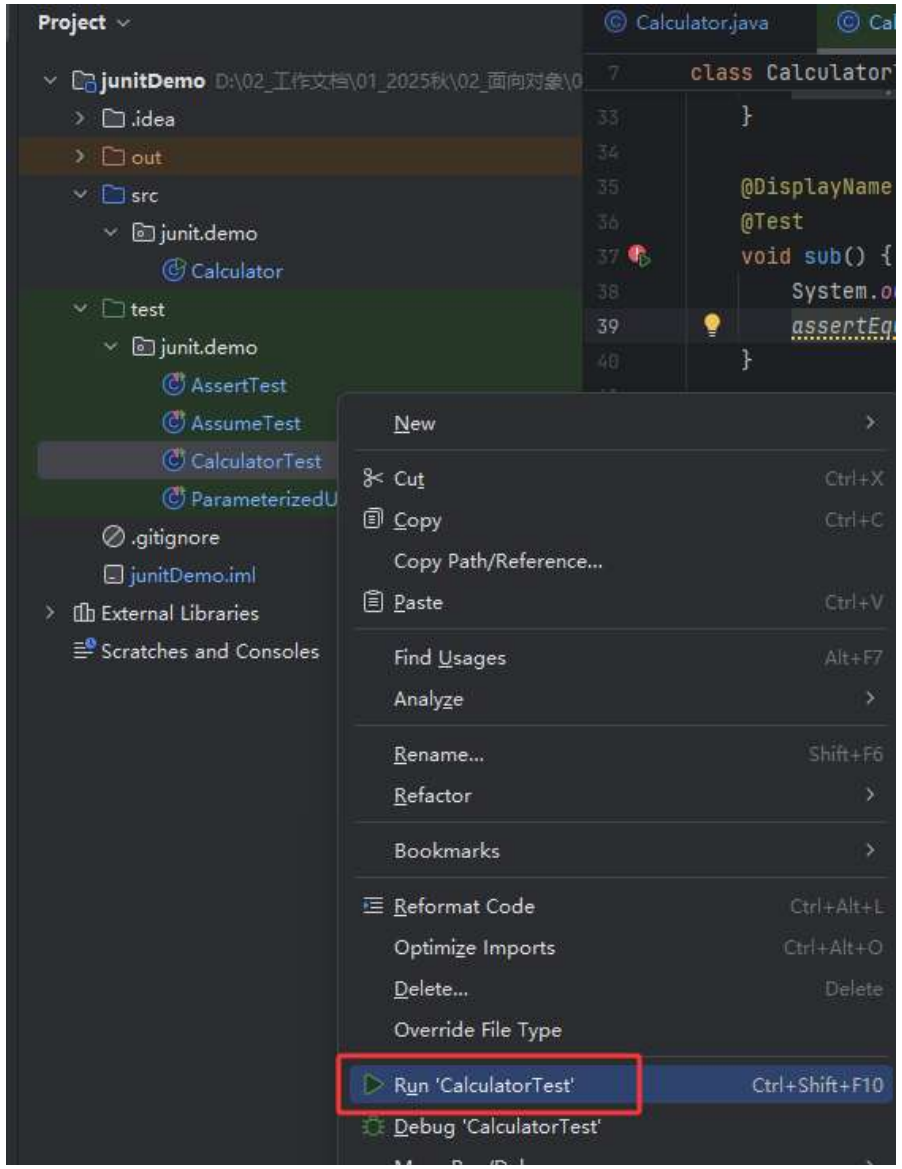
设计测试用例，编写单元测试代码，修改详见指导书4.2节。



实验步骤：如何用JUnit5进行单元测试

⑥ 运行单元测试代码

右键CalculatorTest类，选择 Run CalculatorTest，即可运行。



实验步骤：如何用JUnit5进行单元测试

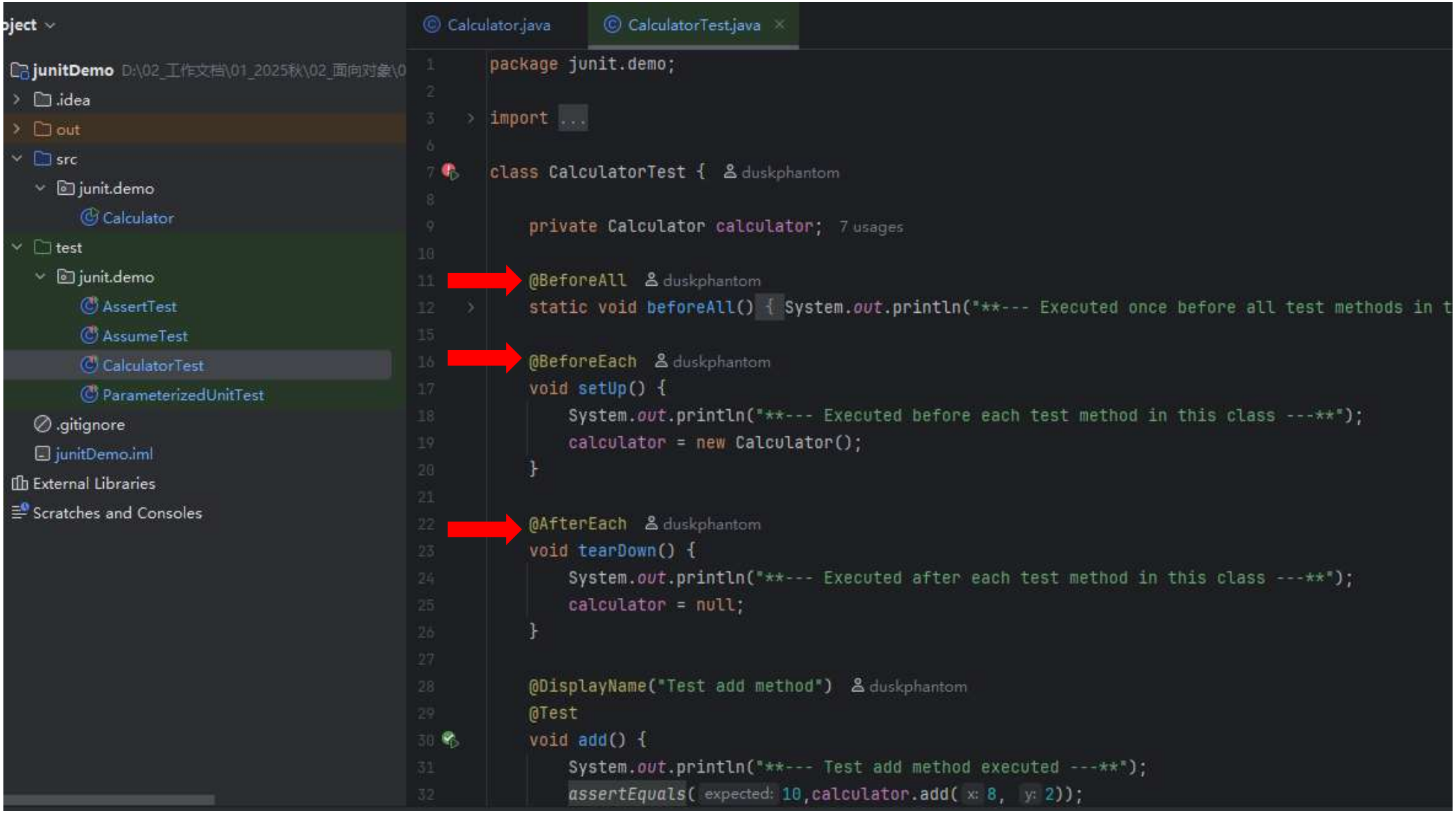
⑦ 根据测试结果分析代码

实验步骤：JUnit5 的常见用法 —— 注解（1）

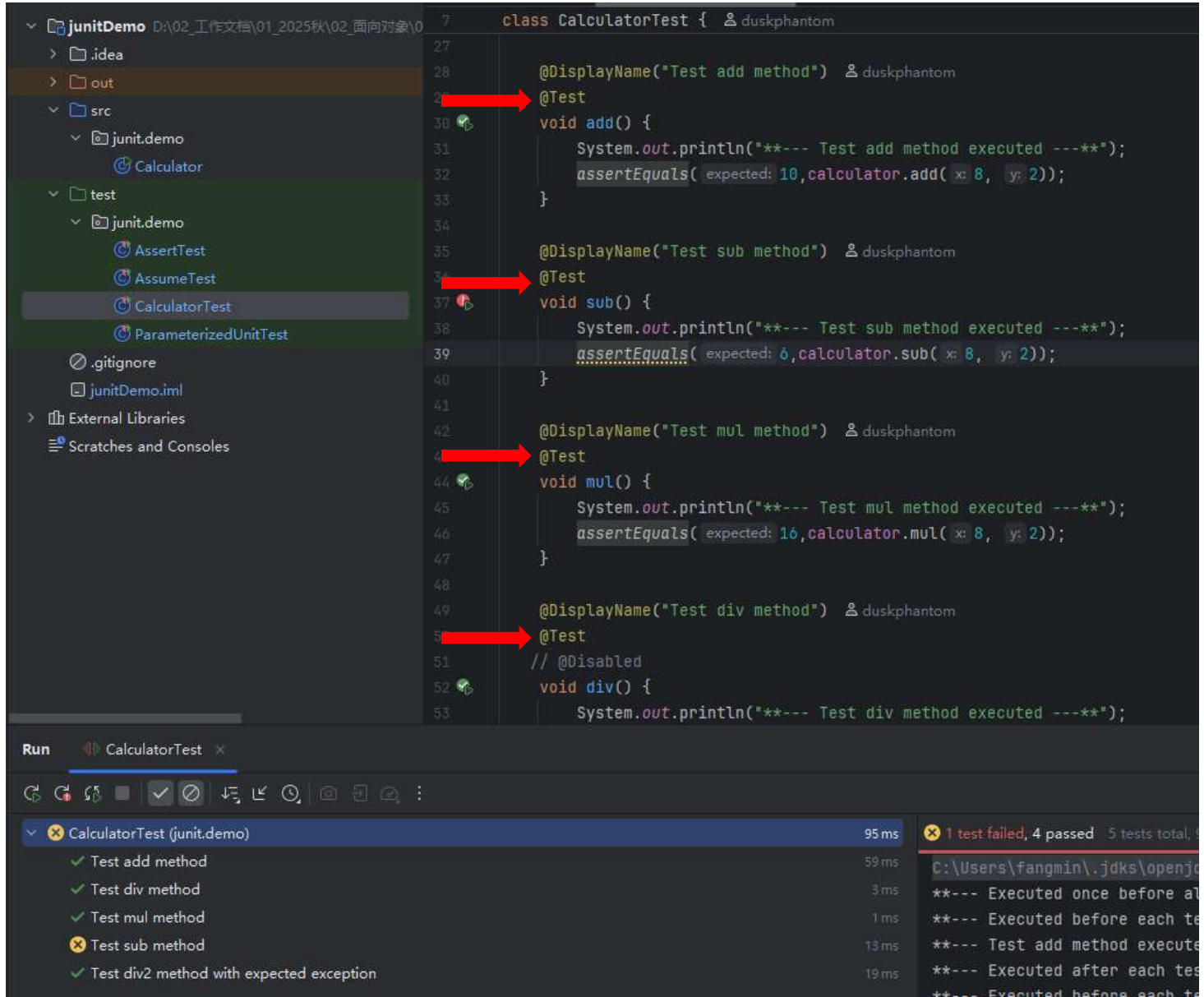
（1）JUnit5注解 (Annotations)

Annotation	Description
@Test	Denotes a test method
@DisplayName	Declares a custom display name for the test class or test method
@BeforeEach	Denotes that the annotated method should be executed before each test method
@AfterEach	Denotes that the annotated method should be executed after each test method
@BeforeAll	Denotes that the annotated method should be executed before all test methods
@AfterAll	Denotes that the annotated method should be executed after all test methods
@Disable	Used to disable a test class or test method
@Nested	Denotes that the annotated class is a nested, non-static test class
@Tag	Declare tags for filtering tests
@ExtendWith	Register custom extensions

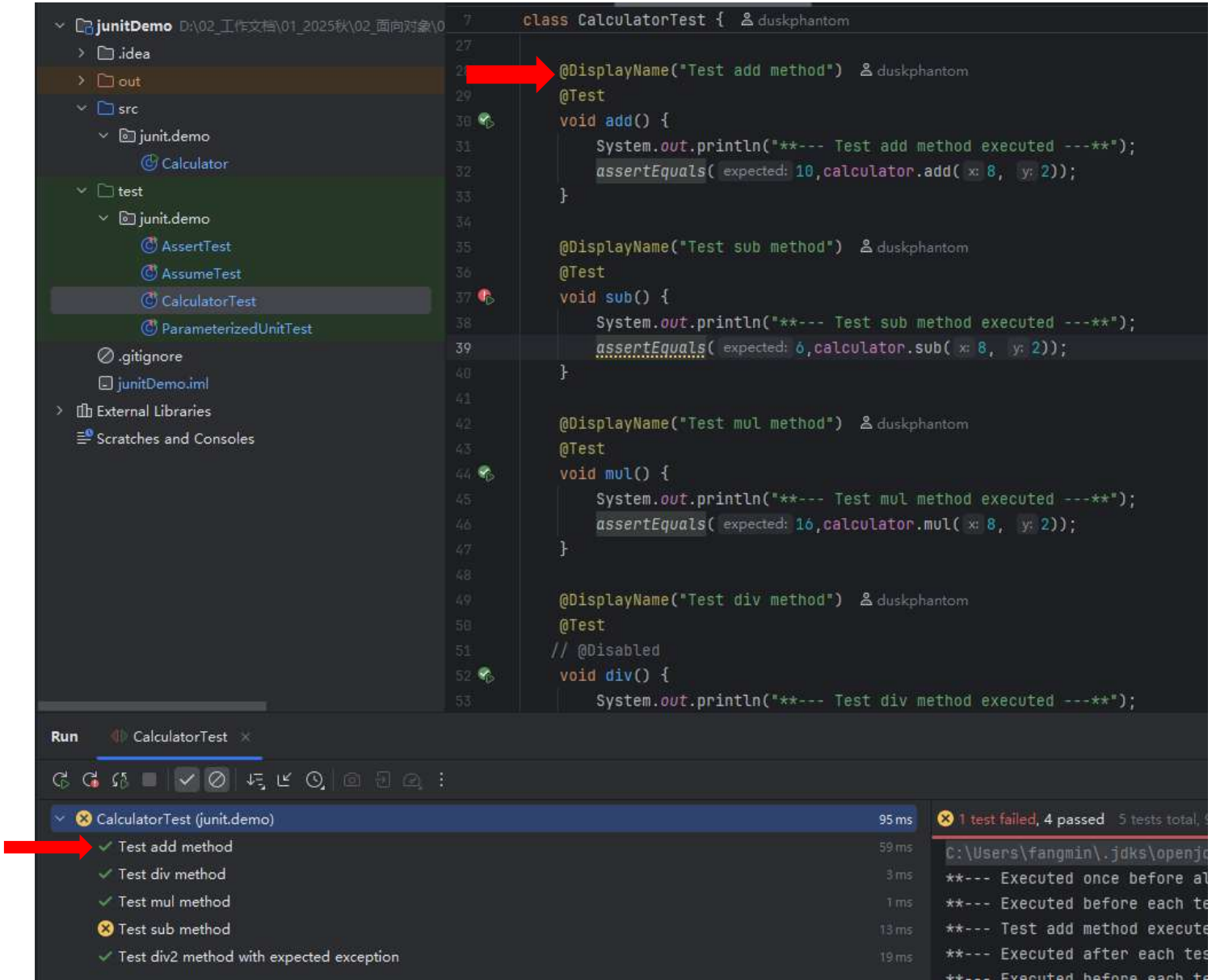
实验步骤：JUnit5 的常见用法



实验步骤：JUnit5 的常见用法



实验步骤：JUnit5 的常见用法



实验步骤：JUnit5 的常见用法

Project

junitDemo D:\02_工作文档\01_2025秋\02_面向对象\0
- .idea
- out
- src
  - junit.demo
    - Calculator
- test
  - junit.demo
    - AssertTest
    - AssumeTest
    - CalculatorTest
    - ParameterizedUnitTest
- .gitignore
- junitDemo.iml

External Libraries

Scratches and Consoles

Calculator.java | CalculatorTest.java

```
7   class CalculatorTest {  duskphantom *
44      void mul() {
45          System.out.println("**--- Test mul method executed ---**");
46          assertEquals( expected: 16,calculator.mul( x: 8,  y: 2));
47      }
48
49      @DisplayName("Test div method")  duskphantom
50      @Test
51      // @Disabled
52      void div() {
53          System.out.println("**--- Test div method executed ---**");
54          assertEquals( expected: 4,calculator.div( x: 8,  y: 2));
55      }
56
57      @Disabled  duskphantom
58      @DisplayName("Test div2 method with expected exception")
59      @Test
60      void div2() {
61          System.out.println("**--- Test div2 method executed ---**");
62          Exception exception = assertThrows(ArithmeticException.class, () -> calculator.div2( x: 2,  y: 0));
63          assertEquals( expected: "/ by zero", exception.getMessage());
64          assertTrue(exception.getMessage().contains("zero"));
65      }
66
67      @AfterAll  duskphantom
68      static void afterAll() { System.out.println("**--- Executed once after all test methods in this class ---*
71
72  }
```

Run | CalculatorTest

- CalculatorTest (junit.demo) 56 ms
  - Test add method 42 ms
  - Test div method 3 ms
  - Test mul method 3 ms
  - Test sub method 8 ms
  - Test div2 method with expected exception

1 test failed, 3 passed, 1 ignored 5 tests total, 56 ms

```
void junit.demo.CalculatorTest.div2() is @Disabled
```

实验步骤：JUnit5 的常见用法 —— 断言（1）

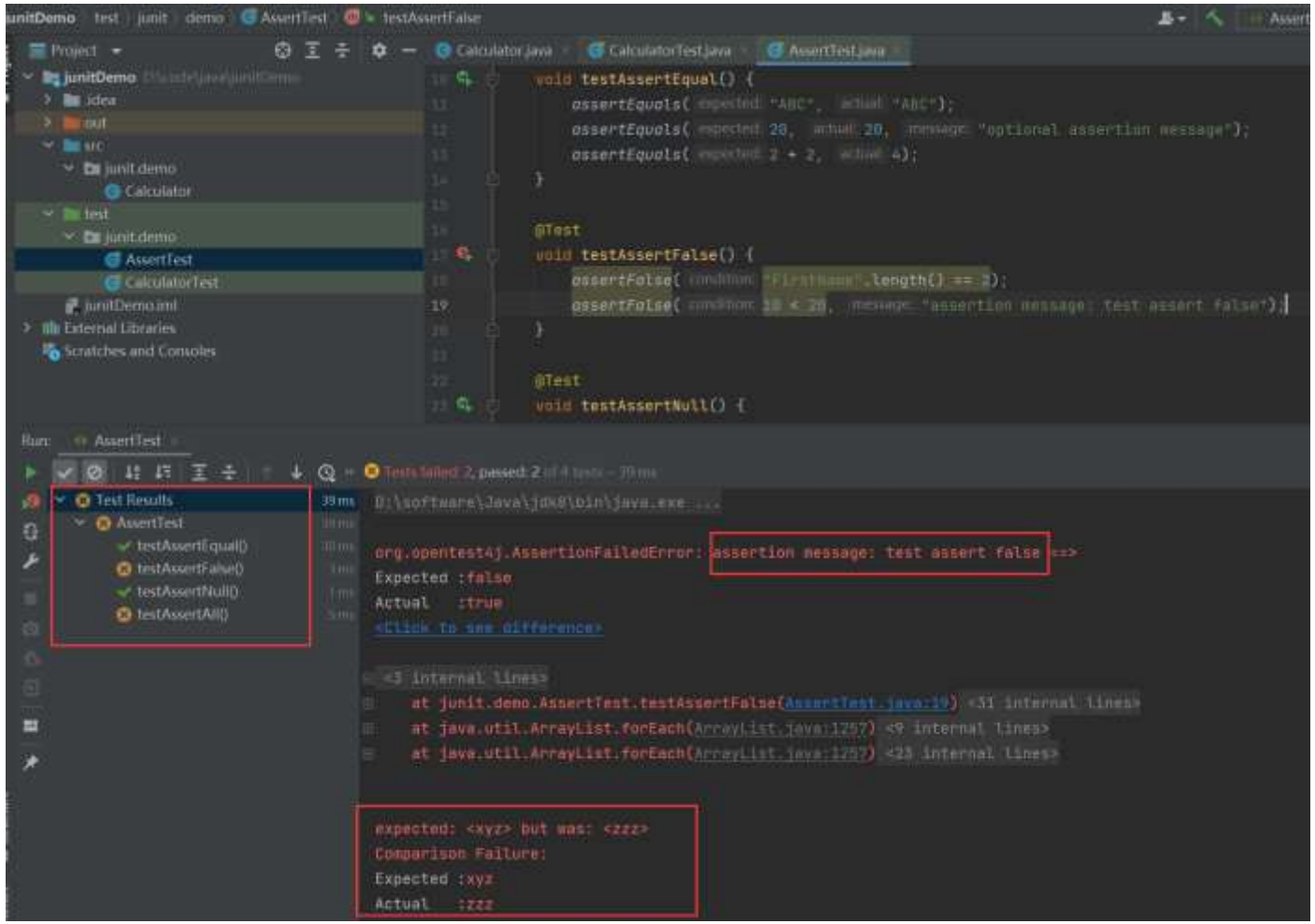
（2）JUnit5断言 (Assertions)

必须使用断言将每个测试方法的条件评估为true，以便测试可以继续执行。

Assertion	Description
assertEquals(expected, actual)	Fails when expected does not equal actual
assertFalse(expression)	Fails when expression is not false
assertNull(actual)	Fails when actual is not null
assertNotNull(actual)	Fails when actual is null
assertAll()	Group many assertions and every assertion is executed even if one or more of them fails
assertTrue(expression)	Fails if expression is not true
assertThrows()	Class to be tested is expected to throw an exception

实验步骤：JUnit5 的常见用法 —— 断言（2）

```
public class AssertTest {
    @Test
    void testAssertEqual() {
        assertEquals("ABC", "ABC");
        assertEquals(2 + 2, 4);
    }
    @Test
    void testAssertFalse() {
        assertFalse("FirstName".length() == 2);
        assertFalse(10 < 20, "assertion message:
                              test assert false");
    }
    @Test
    void testAssertNull() {
        String str1 = null;
        String str2 = "abc";
        assertNull(str1);
        assertNotNull(str2);
    }
...
}
```



实验步骤：JUnit5 的常见用法 —— 假设（1）

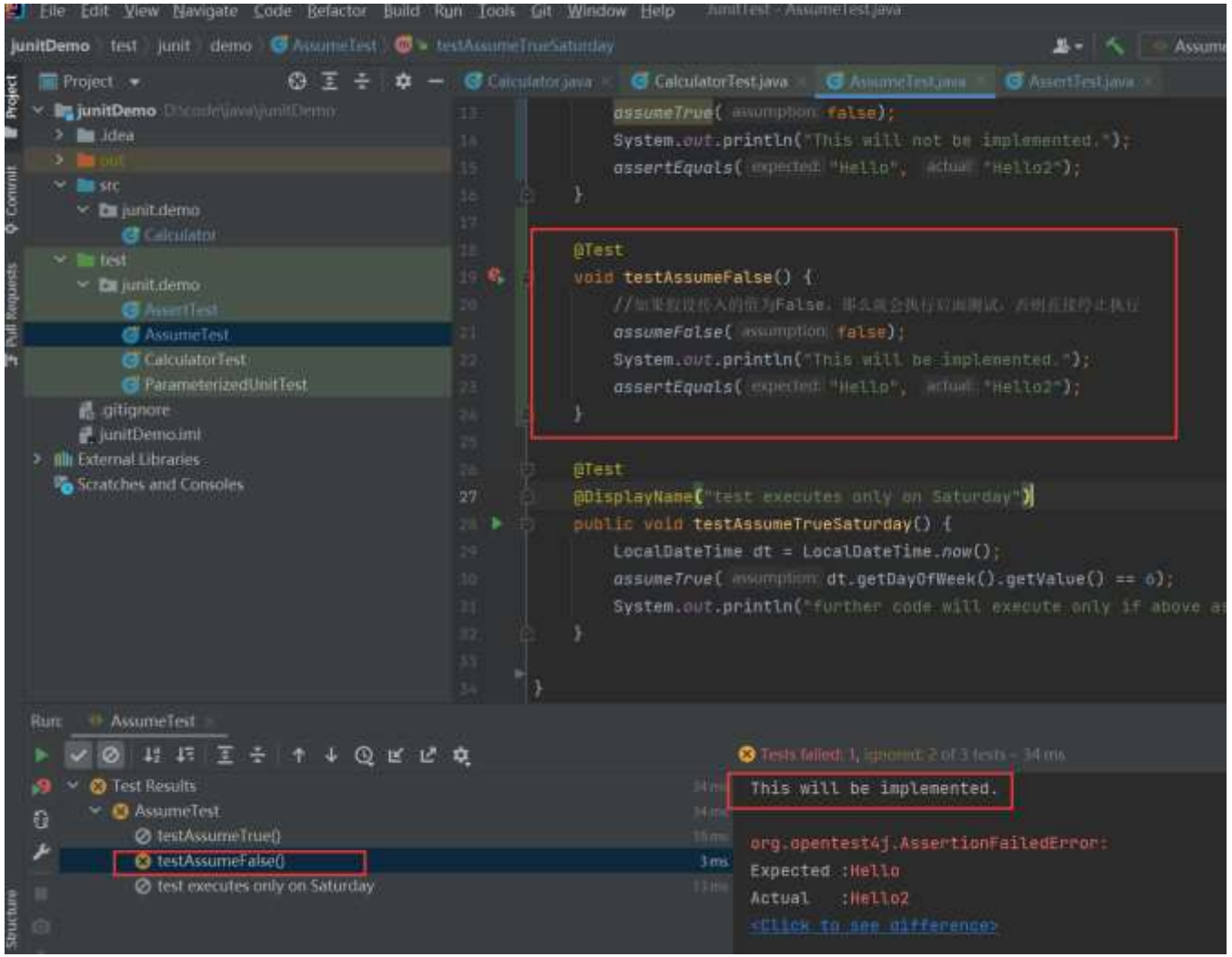
（3）JUnit5假设 (Assumptions)

仅在满足指定条件时执行测试，否则测试将中止。

Assumptions	Description
assumeTrue	Execute the body of lamda when the positive condition hold else test will be skipped
assumeFalse	Execute the body of lamda when the negative condition hold else test will be skipped
assumingThat	Portion of the test method will execute if an assumption holds true and everything after the lambda will execute irrespective of the assumption in assumingThat() holds

实验步骤：JUnit5 的常见用法 —— 假设（2）

```
public class AssumeTest {
    @Test
    void testAssumeFalse() {
      assumeFalse(false);
      System.out.println("This will be
                                  implemented.");
      assertEquals("Hello", "Hello2");
    }

    @Test
    @DisplayName("executes only on Saturday")
    public void testAssumeTrueSaturday() {
       LocalDateTime dt = LocalDateTime.now();
      assumeTrue(dt.getDayOfWeek().getValue()
                          == 6);
       System.out.println("further code will
       execute only if above assumption holds
       true");
    }
...
}
```



实验步骤：JUnit5 的常见用法

假设 VS 断言

Assume 不满足 → 跳过；

假设：检查**前提条件/环境**是否满足（系统属性、操作系统、日期等）；不满足就**中止并标记为 Skipped**，不算失败。

Assert 不满足 → 失败

断言：验证**业务行为/结果**是否符合预期；不满足则**失败**，用于发现缺陷。

实验步骤：JUnit5 的常见用法 —— 测试异常（1）

（4）JUnit5测试异常 (Test Exception)

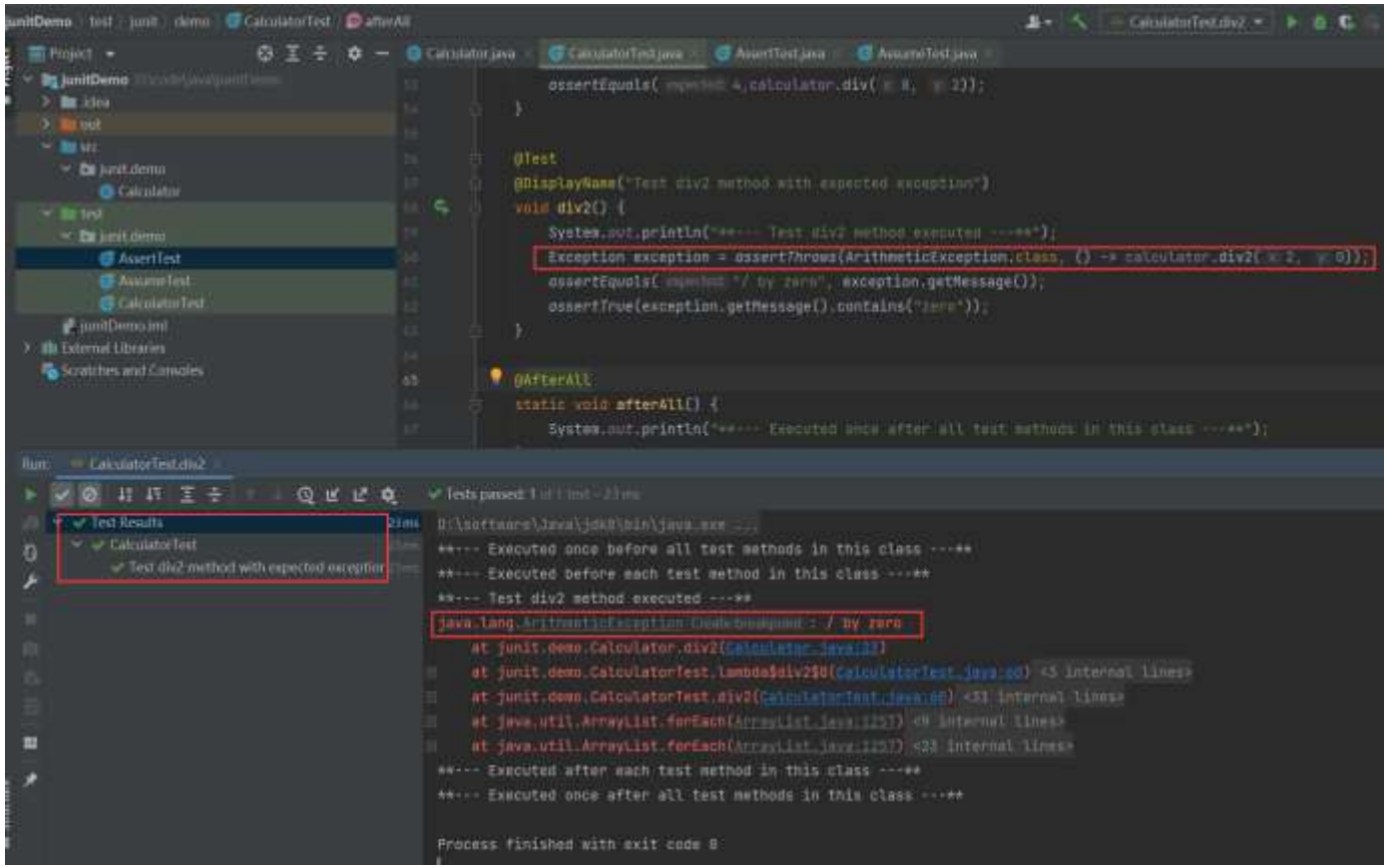
在某些情况下，期望方法在特定条件下引发异常。 如果给定方法未引发指定的异常，则assertThrows将使测试失败。

```
public static <T extends Throwable> T assertThrows(Class<T> expectedType, Executable executable)
```

它断言所提供的executable的执行将引发expectedType的异常并返回该异常。

实验步骤：JUnit5 的常见用法 —— 测试异常（2）

```
class CalculatorTest {

private Calculator calculator;
...
    @Test
    @DisplayName("Test div2 method
                 with expected exception")
    void div2() {
        Exception exception =  assertThrows(
                ArithmeticException.class,
                () -> calculator.div2(2, 0));
        assertEquals("/ by zero",
                        exception.getMessage());
        assertTrue(exception.getMessage()
                        .contains("zero"));
    }

}
```



实验步骤：JUnit5 的常见用法 —— 参数测试（1）

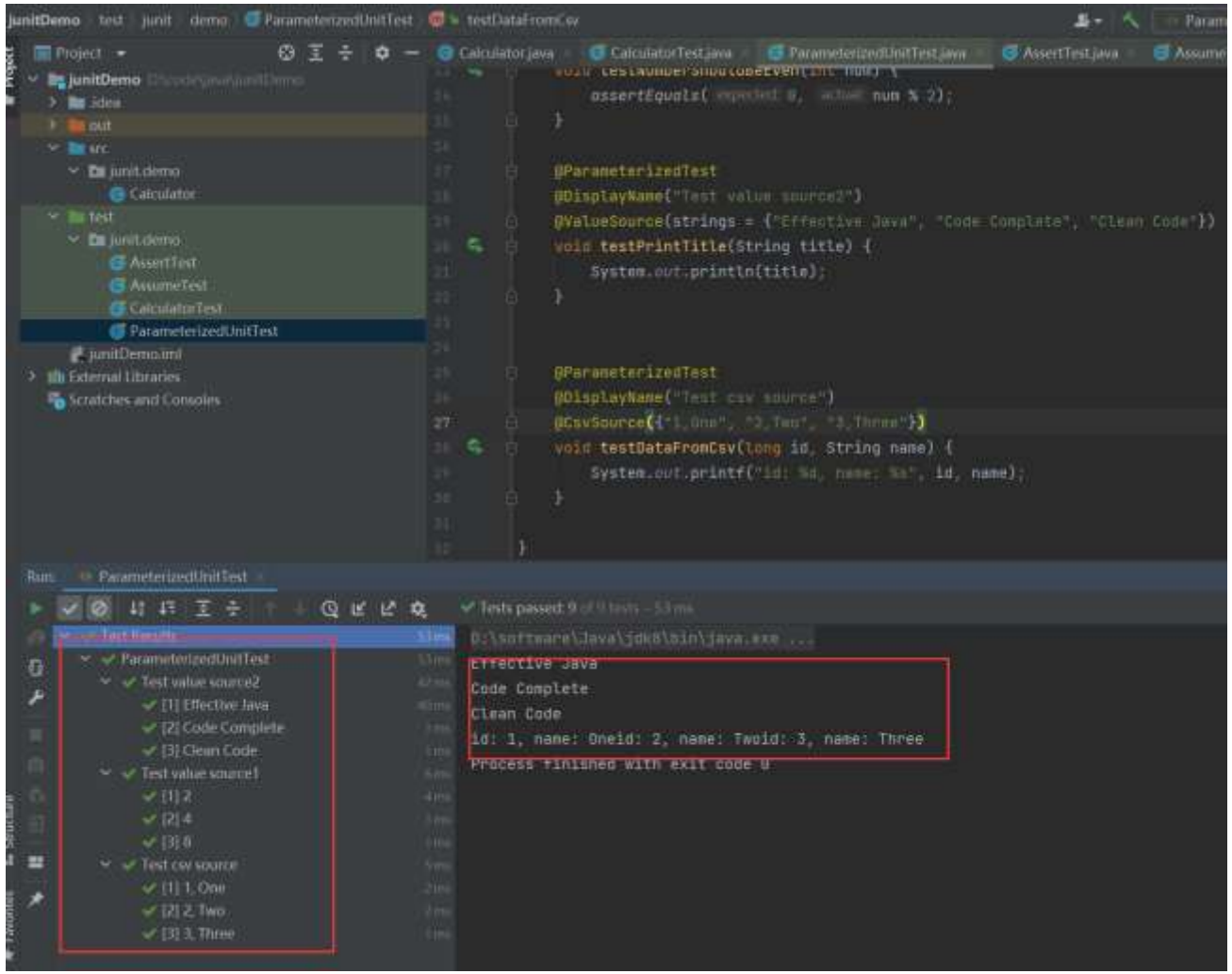
（5）JUnit5参数测试 (Parameterized Tests)

@ParameterizedTest 作为参数化测试的必要注解，替代了 @Test 注解。任何一个参数化测试方法都需要标记上该注解。

基本数据源测试： @ValueSource @ValueSource 是 JUnit 5 提供的最简单的数据参数源，支持 Java 的八大基本类型、字符串和Class，使用时赋值给注解上对应类型属性，以数组方式传递。
CSV 数据源测试： @CsvSource 通过 @CsvSource 可以注入指定 CSV 格式 (comma-separated-values) 的一组数据，用每个逗号分隔的值来匹配一个测试方法对应的参数。

实验步骤：JUnit5 的常见用法 —— 参数测试（2）

```
public class ParameterizedUnitTest {

    @ParameterizedTest
    @DisplayName("Test value source1")
    @ValueSource(ints = {2, 4, 8})
    void testNumberShouldBeEven(int num) {
        assertEquals(0, num % 2);
    }
...

    @ParameterizedTest
    @DisplayName("Test csv source")
    @CsvSource({"1,One", "2,Two", "3,Three"})
    void testDataFromCsv(long id, String name) {
        System.out.printf("id: %d, name: %s", id,
        name);
    }

}
```



实验步骤：测试用例示例

用例编号	1			
待测试类及方法	HeroAircraft.crash			
测试类及方法	HeroAircraftTest.crash			
前提条件（如有）	创建指定坐标的道具或敌机			
用例描述	测试步骤	期望结果	实际输出	测试结果
测试两个飞行物体的碰撞检测正否正常。	1. 英雄机初始化，指定坐标（x, y）（具体值） 2. 创建火力道具，指定坐标（x, y）（具体值，可以相撞） 3. 调用英雄机的 crash 方法，传入火力道具 4. 判断是否检测到碰撞	Crash 返回 true	True	Pass

本次迭代开发目标（1）

任务清单
(CheckList)

UML类图设计

① 结合实例，按照规范绘制DAO模式的结构类图

设计模式应用

② 完成代码重构，采用 DAO 模式实现排行榜数据的持久化存储功能

文件读写

③ 得分数据需存储在文件中，且每个游戏难度对应单独的存储文件，确保数据分离

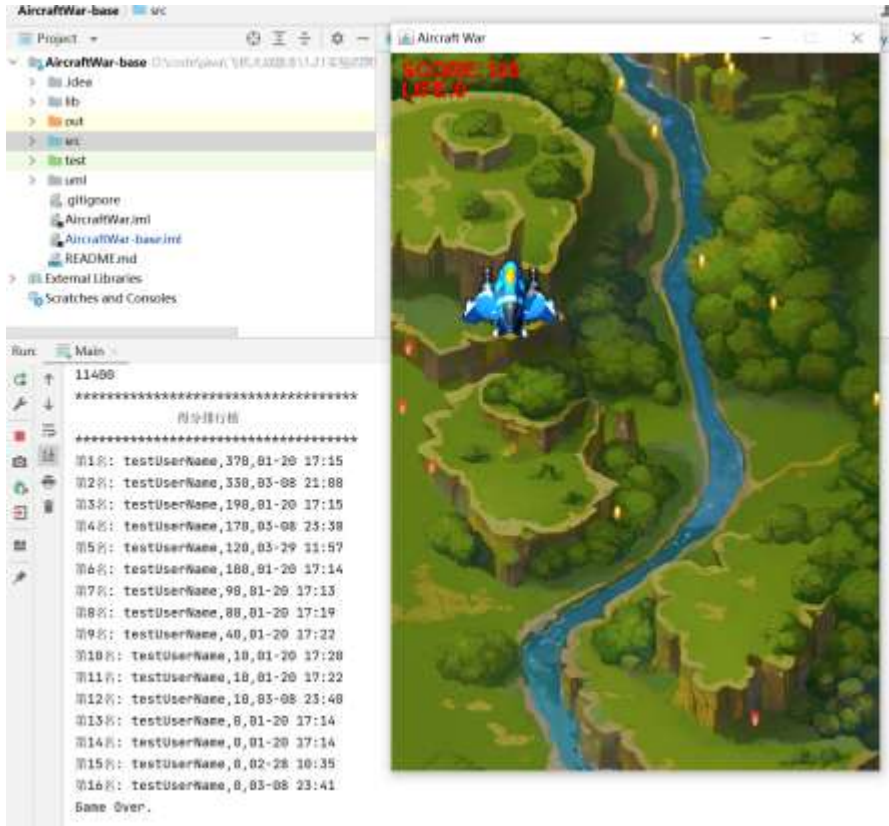
本次迭代开发目标（2）

任务清单
(CheckList)

排行榜

④ 每局游戏结束后，将当局游戏得分、时间插入数据记录；并展示当前难度下的最新总分排行榜，榜单需包含名次、玩家名、得分及时间

注：本次实验仅需在游戏结束后记录该局分数，使用默认玩家名即可；排行榜无需实现界面展示及玩家交互功能，在控制台打印输出。



本次迭代开发目标（2）

任务清单
(CheckList)

单元测试

⑤ 对英雄机类进行单元测试，至少选择3 个方法（含其父类方法）作为测试对象，设计合理测试用例并完成测试

同学们，请开始实验吧！

THANK YOU